OS banana problem unit 4

SRN: PES1UG24CS697
SECTION: L
NAME : UTTAM K R

Phase 1: Object Storage Foundation

Screenshot 1A: Output of `./test_objects` showing all tests passing

```
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
```

Screenshot 1B: `find .pes/objects -type f` showing the sharded directory structure.

```
PASS: deduplication
PASS: integrity check

Open file in editor (ctrl + click) 4CS697-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
```

Phase 2: Tree Objects

Screenshot 2A: Output of `./test_tree` showing all tests passing.

```
gcc -Wall -Wextra -O2 -c index.c -o index.o
^[[Agcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
Uttam K R (3 hours ago)  Ln 15, Col 20  Spaces: 4  UTF-8  LF  { } C
```

Screenshot 2B: Pick a tree object from `find .pes/objects -type f` and run `xxd .pes/objects/XX/YYY... | head -20` to show the raw binary format.

```
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337
b46a1af8e11ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
Uttam K R (3 hours ago)  Ln 15, Col 20  Spaces: 4  UTF-8  LF  { } C
```

Phase 3: The Index (Staging Area)

Screenshot 3A: Run `./pes init`, `./pes add file1.txt file2.txt`, `./pes status` — show the output.

```
./pes status
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ rm -rf .pes file1.txt file2.txt
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  commit.h
  untracked:  tree.h
  untracked:  index.c
  untracked:  test_tree
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  test_objects.c

vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
Uttam1916 (11 minutes ago)  Ln 32, Col 45  Spaces: 4  UTF-8  { } C
```

Screenshot 3B: `cat .pes/index` showing the text-format index with your entries.

```
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776331898 6 file1.
txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776331898 6 file2.
txt
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
Uttam1916 (11 minutes ago)  Ln 32, Col 45  Spaces: 4  UTF-8  { } C
```

Phase 4: Commits and History

Screenshot 4A: Output of `./pes log` showing three commits with hashes, authors, timestamps, and messages.

```
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ ./pes log
commit 648e3205a272a1cdc55eb8d5957a30832d52fe466d6cdad8ab
236d7d8b93180a
Author: Test User <PES1UG24CS697>
Date:   1776351684

    Add farewell

commit 6f376fe7ddd5a5c5d8f00d313d1bb31f49a78464bc9d96c9ca
a9f1af47e6186c
Author: Test User <PES1UG24CS697>
Date:   1776351684

    Add worldell

commit 433273dd47ea045811f41406961279ec769038ec1966d78bd5
40fbc45adadff4
Author: Test User <PES1UG24CS697>
Date:   1776351683

    Initial committial comQ

vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
Uttam1916 (5 hours ago)   Ln
```

Screenshot 4B: `find .pes -type f | sort` showing object store growth after three commits.

```
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ find .pes -type f
 | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e6806
0c265ca21cbd6c05e5f43
.pes/objects/43/3273dd47ea045811f41406961279ec769038ec196
6d78bd540fbc45adadff4
.pes/objects/64/8e3205a272a1cdc55eb8d5957a30832d52fe466d6
cdad8ab236d7d8b93180a
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a
1a52d3d5f4aa8007b5c39
.pes/objects/6f/376fe7ddd5a5c5d8f00d313d1bb31f49a78464bc9
d96c9caa9f1af47e6186c
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb
383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c20551
30c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a86
7d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84d
f834438880e13a68652bc
.pes/refs/heads/main
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
Uttam1916 (5 hours ago)    Ln
```

Screenshot 4C: `cat .pes/refs/heads/main` and `cat .pes/HEAD` showing the reference chain.

```
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$ echo "=== .pes/refs/heads/main ===" && c
at .pes/refs/heads/main && echo "" && echo "=== .pes/HEAD ===" && cat .pes/HEAD
=== .pes/refs/heads/main ===
648e3205a272a1cdc55eb8d5957a30832d52fe466d6cdad8ab236d7d8b93180a

=== .pes/HEAD ===
ref: refs/heads/main
vboxuser@ubunu:~/PES1UG24CS697-pes-vcs$
```

Phase 5 & 6: Analysis-Only Questions

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout — what files

need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout, you mainly change where HEAD points. You update the HEAD file so it now refers to the new branch, then read the commit hash stored in that branch file. That commit gives you the snapshot (tree) of the project.

After that, the working directory must be updated to match that snapshot. Files may need to be created, deleted, or overwritten so everything matches the new branch. The index should also be updated to reflect this state.

It is complex because you are modifying the whole working directory, not just one pointer. You have to handle uncommitted changes, avoid data loss, and correctly update many files and directories.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

To detect this, you compare the working directory with the index. For each tracked file, compute its current hash and compare it with the hash stored in the index. If they are different, the file is modified and not committed.

Then compare the index version with the version in the target branch’s tree. If the file is modified in the working directory and also differs in the target branch, it means switching would overwrite changes. In that case, checkout should refuse.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In detached HEAD, new commits are still created normally, but they are not attached to any branch. So there is no branch pointing to them, and they can become unreachable later.

To recover them, the user can create a new branch pointing to that commit. For example, by creating a branch at the current commit, those commits become part of normal history again and won’t be lost.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Start from all branch heads and mark everything reachable. For each branch, take its commit and recursively visit its parent commits, their trees, and all blobs and subtrees. While doing this, store all visited hashes in a hash set so you don’t process the same object twice.

After marking is done, go through all objects in the object store and delete those that are not in the reachable set.

A hash set is best for tracking reachable objects because it gives fast lookup and avoids duplicates.

For 100,000 commits, you would visit roughly all commits plus their trees and blobs. So it would be on the order of a few hundred thousand to maybe a million objects depending on the project size.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

It is dangerous because GC might delete objects that are still being created or not yet fully linked into the repository.

For example, during a commit, new objects like blobs and trees are written first, and only at the end the branch pointer is updated. If GC runs in between, it may not see these new objects as reachable yet and delete them. Then the commit will point to missing objects, causing corruption.

Git avoids this by making object creation and reference updates safe. It uses temporary files and atomic renames so objects appear fully written. Also, GC runs with safety rules, like not deleting very recent objects and coordinating with locks, so it does not remove objects that might still be in use.